

COMPLETE PROJECT DOCUMENTATION

Save This Entire Message - It Has EVERYTHING

PROJECT: RAG Evaluation Pipeline

Type: Full-Stack AI Web Application

Purpose: Production-grade Document Q&A System with Automated Evaluation

Live URL: <https://rag-evaluation-pipeline.vercel.app>

API URL: <https://rag-evaluation-pipeline.onrender.com>

API Docs: <https://rag-evaluation-pipeline.onrender.com/docs>

GitHub: <https://github.com/Karthik26544/rag-evaluation-pipeline>

PROJECT STRUCTURE

text

rag-evaluation-pipeline/

```
|
|
| └─ backend/          (Python FastAPI)
|
|   └─ main.py          (FastAPI app entry point)
|   └─ requirements.txt  (Python packages)
|   └─ .env              (Environment variables - NOT in git)
|   |
|   | └─ routers/
|   |   └─ __init__.py
|   |   └─ documents.py  (Upload, List, Delete documents)
|   |   └─ queries.py    (Ask questions, History, Caching)
|   |   └─ evaluation.py (Run evaluations, Dashboard data)
|   |   └─ auth.py       (Register, Login, JWT tokens)
|   |   └─ admin.py      (Admin stats, Users, Analytics, Export)
|   |   └─ feedback.py   (Submit, View, Stats for feedback)
|   |
|   |
```

- | ├── services/
- | | ├── __init__.py
- | | ├── document_processor.py (PDF/DOCX/TXT extraction + chunking)
- | | ├── vector_store.py (Qdrant + Gemini embeddings)
- | | ├── hybrid_search.py (BM25 + Vector fusion)
- | | ├── query_processor.py (Gemini LLM for answers)
- | | ├── reranker.py (Cohere cross-encoder reranking)
- | | ├── cache_service.py (SHA256 query caching)
- | | ├── auth_service.py (JWT + bcrypt password hashing)
- | | └── auth_middleware.py (Protected route middleware)
- | |
- | └── temp_uploads/ (Temporary file storage)
- |
- └── frontend/ (React + TypeScript)
 - | ├── public/
 - | ├── .env (API URL)
 - | ├── .env.production (CI=false for Vercel builds)
 - | ├── package.json
 - | ├── tailwind.config.js
 - | |
 - | └── src/
 - | ├── App.tsx (Routes + Navigation + Auth)
 - | ├── index.css (Tailwind + Tour dark theme styles)
 - | |
 - | ├── context/
 - | └── AuthContext.tsx (Auth state + Axios interceptors)
 - | |

```

|   ├── components/
|   |   ├── ProtectedRoute.tsx (Auth guard for routes)
|   |   ├── AppTour.tsx   (Interactive guided tour - driver.js)
|   |   └── FeedbackModal.tsx (Star rating feedback popup)
|   |
|   └── pages/
|       ├── LoginPage.tsx   (Email + Password login)
|       ├── RegisterPage.tsx (User registration)
|       ├── UploadPage.tsx  (Document upload with strategy selection)
|       ├── ChatPage.tsx   (Q&A with search/reranker toggles)
|       ├── HistoryPage.tsx (Per-user query history with search)
|       ├── EvaluationPage.tsx (Run evaluation comparisons)
|       ├── DashboardPage.tsx (Evaluation charts)
|       └── AdminDashboard.tsx (Full admin panel with 6 tabs)
|
|── .gitignore
|── README.md

```

TECHNOLOGY STACK

Backend

Technology	Version	Purpose
Python	3.11.9	Runtime
FastAPI	0.115.0	Web framework (like Express but Python)
Uvicorn	0.30.6	ASGI server
PostgreSQL	16	Primary database
Qdrant	Latest	Vector database for embeddings

Technology	Version	Purpose
Google Gemini	gemini-flash-latest	LLM for answers + query rewriting
Gemini Embeddings	gemini-embedding-001	Text to vectors (768 dim locally, 3072 cloud)
Cohere	rerank-english-v3.0	Cross-encoder reranking
Sentence Transformers	BAAI/bge-small-en-v1.5	Local embeddings (384 dim)
LangChain	0.3.7	Text splitting utilities
BM25 (rank-bm25)	0.2.2	Keyword-based search
python-jose	3.3.0	JWT token creation/verification
passlib + bcrypt	1.7.4 / 4.0.1	Password hashing
slowapi	0.1.9	Rate limiting
psycopg2-binary	2.9.9	PostgreSQL driver
pypdf	5.0.1	PDF text extraction
python-docx	1.1.2	DOCX text extraction

Frontend

Technology	Version	Purpose
React	19	UI framework
TypeScript	Latest	Type safety
Tailwind CSS	3.4.1	Utility-first styling
React Router DOM	Latest	Client-side routing
Axios	Latest	HTTP requests + interceptors
Recharts	Latest	Charts (Line, Pie, Bar)
Lucide React	Latest	Icon library
React Hot Toast	Latest	Toast notifications
driver.js	Latest	Interactive guided tour

Deployment

Service	Purpose	Tier
Vercel	Frontend hosting	Free
Render	Backend hosting	Free
Neon	Cloud PostgreSQL	Free
Qdrant Cloud	Cloud vector database	Free
UptimeRobot	Keep Render warm (ping every 5 min)	Free

DATABASE SCHEMA (7 Tables)

SQL

1. users

- id (UUID, primary key)
- name (VARCHAR)
- email (VARCHAR, unique)
- password_hash (VARCHAR)
- is_admin (BOOLEAN, default false)
- created_at (TIMESTAMP)
- last_login (TIMESTAMP)
- total_queries (INTEGER)
- total_documents (INTEGER)

2. documents

- id (UUID, primary key)
- user_id (UUID, FK → users, CASCADE delete)
- filename (VARCHAR)
- file_type (VARCHAR)
- total_chunks (INTEGER)
- chunking_strategy (VARCHAR)

- upload_time (TIMESTAMP)
- status (VARCHAR)

3. chunks

- id (UUID, primary key)
- document_id (UUID, FK → documents, CASCADE delete)
- content (TEXT)
- chunk_index (INTEGER)
- chunk_size (INTEGER)
- chunking_strategy (VARCHAR)
- metadata (JSONB)
- created_at (TIMESTAMP)

4. queries

- id (UUID, primary key)
- user_id (UUID, FK → users, SET NULL)
- question (TEXT)
- rewritten_question (TEXT)
- answer (TEXT)
- sources (JSONB)
- search_type (VARCHAR)
- chunking_strategy (VARCHAR)
- latency_ms (INTEGER)
- confidence_score (FLOAT)
- created_at (TIMESTAMP)

5. evaluations

- id (UUID, primary key)

- evaluation_name (VARCHAR)
- chunking_strategy (VARCHAR)
- search_type (VARCHAR)
- faithfulness (FLOAT)
- answer_relevancy (FLOAT)
- total_questions (INTEGER)
- avg_latency_ms (INTEGER)
- evaluated_at (TIMESTAMP)

6. query_cache

- id (UUID, primary key)
- question_hash (VARCHAR(64), unique)
- question (TEXT)
- answer (TEXT)
- sources (JSONB)
- confidence (FLOAT)
- search_type (VARCHAR)
- hit_count (INTEGER, default 1)
- created_at (TIMESTAMP)
- last_hit_at (TIMESTAMP)

7. feedback

- id (UUID, primary key)
- user_id (UUID, FK → users, CASCADE delete)
- rating (INTEGER, 1-5)
- comment (TEXT)
- category (VARCHAR)
- created_at (TIMESTAMP)

🔑 ALL API ENDPOINTS

Auth Routes (/api/auth)

text

POST /api/auth/register - Create new user account

POST /api/auth/login - Login and get JWT token

GET /api/auth/me - Get current user info (requires auth)

Document Routes (/api/documents)

text

POST /api/documents/upload - Upload and process document (rate limited: 5/min)

GET /api/documents/list - Get all uploaded documents

DELETE /api/documents/{id} - Delete a document

Query Routes (/api/queries)

text

POST /api/queries/ask - Ask a question (rate limited: 10/min)

GET /api/queries/history - Get all queries (public)

GET /api/queries/my-history - Get current user's queries (requires auth)

DELETE /api/queries/my-history/{id} - Delete a query (requires auth)

Evaluation Routes (/api/evaluation)

text

POST /api/evaluation/run - Run evaluation across strategies

GET /api/evaluation/history - Get past evaluations

GET /api/evaluation/dashboard - Get dashboard chart data

Feedback Routes (/api/feedback)

text

POST /api/feedback/submit - Submit feedback (requires auth)

GET /api/feedback/my-feedback - Get user's own feedback (requires auth)

GET /api/feedback/all - Get all feedback (admin only)

GET /api/feedback/stats - Get feedback statistics (admin only)

DELETE /api/feedback/{id} - Delete feedback (admin only)

Admin Routes (/api/admin) - All require admin role

text

GET /api/admin/stats - Platform statistics (12 metrics)

GET /api/admin/users - All users with activity stats

GET /api/admin/users/{id}/details - Deep dive into single user

POST /api/admin/users/{id}/toggle-admin - Toggle admin status

DELETE /api/admin/users/{id} - Delete user

GET /api/admin/queries/recent - Recent queries with user info

GET /api/admin/analytics/growth - 30-day growth + distributions + activity feed

GET /api/admin/export/users - Download users CSV

GET /api/admin/export/queries - Download queries CSV

POST /api/admin/cache/clear - Clear all cached queries

AUTHENTICATION FLOW

text

1. User registers (name, email, password)
2. Password hashed with bcrypt (passlib)
3. User stored in PostgreSQL
4. JWT token generated (contains userId, email, isAdmin)
5. Token expires in 7 days
6. Frontend stores token + user in localStorage
7. Axios interceptor adds "Authorization: Bearer <token>" to EVERY request
8. Backend middleware verifies token on protected routes
9. Admin routes additionally check isAdmin flag
10. 401 responses auto-clear localStorage (axios response interceptor)

RAG PIPELINE FLOW

text

User Question

↓

Query Rewriting (Gemini expands ambiguous queries)

↓

Retrieval (top 20 candidates)

├— Vector Search (Gemini embeddings → Qdrant)

├— BM25 Keyword Search (rank-bm25)

└— Hybrid Fusion ($\alpha=0.7$ weighted combination)

↓

Cross-Encoder Reranking (Cohere rerank-v3.0)

- Inputs: top 20 chunks

- Outputs: top 5 most relevant

↓

Answer Generation (Gemini Flash)

- Context: top 5 chunks

- Instructions: cite sources, don't hallucinate

↓

Answer + Source Citations + Confidence Score

↓

Cache (if confidence > 0.5, SHA256 hash of question)

THREE CHUNKING STRATEGIES

Strategy	How It Works	Best For
Fixed	500 chars with 50-char overlap	Simple structured docs

Strategy	How It Works	Best For
Recursive	Respects paragraph/sentence boundaries with 100-char overlap	Most general documents (RECOMMENDED)
Sentence	Groups semantically related sentences up to 600 chars	Q&A style content

THREE SEARCH METHODS

Method	How It Works	Trade-off
Vector	Gemini embeddings → cosine similarity in Qdrant	Great for concepts, weak for exact text
BM25	Term frequency ranking (rank-bm25)	Great for exact keywords, weak for synonyms
Hybrid	$\alpha \times \text{vector} + (1-\alpha) \times \text{bm25}$ where $\alpha=0.7$	Best overall quality (DEFAULT)

CROSS-ENCODER RERANKER

text

Input: 20 candidate chunks from retrieval

Process: Cohere rerank-english-v3.0 scores each (query, chunk) pair

Output: Top 5 chunks ranked by relevance

Improvement: ~15-20% precision boost over baseline retrieval

Cost: Free (1000 req/month on Cohere trial)

CACHING SYSTEM

text

How it works:

1. Question arrives
2. SHA256 hash computed from (question + search_type)
3. Check query_cache table for matching hash
4. If found (cache HIT):
 - Return cached answer immediately

- Latency: ~50ms (vs ~3000ms for fresh query)
- Increment hit_count

5. If not found (cache MISS):

- Run full RAG pipeline
- Store result in cache (if confidence > 0.5)
- TTL: 24 hours

EVALUATION PIPELINE (The Gold Feature)

text

How it works:

1. Admin provides test questions with expected (ground truth) answers
2. System runs each question through:
 - Multiple chunking strategies (fixed, recursive, sentence)
 - Multiple search methods (vector, hybrid)
3. For each combination:
 - Generate answer using RAG pipeline
 - Score faithfulness (0-1): Does answer stay true to retrieved context?
 - Score relevancy (0-1): Does answer address the question?
 - Measure latency (ms)
4. Results compared in dashboard with bar charts
5. Best combination identified automatically

Scoring method: LLM-as-Judge (Gemini scores each answer)

COMPLETE FEATURE LIST

User Features

text

1. Register/Login with JWT authentication

2. Upload documents (PDF, DOCX, TXT, Markdown)
3. Choose chunking strategy (Fixed/Recursive/Sentence)
4. Ask questions with multiple search methods (Vector/BM25/Hybrid)
5. Toggle query rewriting (Gemini expands queries)
6. Toggle cross-encoder reranking (Cohere rerank-v3.0)
7. Toggle response caching (SHA256-based)
8. View source citations with relevance scores
9. See confidence score per answer
10. See latency per query
11. See cached badge on repeated queries
12. See reranked badge when reranker active
13. View personal query history with search
14. Delete individual queries from history
15. Submit star rating feedback (1-5 stars)
16. Choose feedback category (General/UI/Performance/Feature/Bug/AI Quality)
17. Interactive guided tour (driver.js, 10 steps)
18. Restart tour anytime with help button
19. Cold start notification (shows message when Render wakes up)
20. Rate limiting (10 queries/min, 5 uploads/min)

Admin Features

text

1. 12 stat cards organized in 3 rows:
 - User Metrics: Total Users, Active Today, Avg Rating, Feedback Count
 - Query Performance: Total Queries, Avg Latency, Confidence, Documents
 - Cost Optimization: Est. Cost, Cache Savings, Cache Entries, Evaluations
2. 6 tabs: Overview, Activity, Users, Queries, Feedback, Cache
3. 4 charts: User Growth, Query Growth, Search Methods Pie, Rating Distribution
4. Real-time activity feed (users registered, queries asked, docs uploaded)

5. User management (search, view details, toggle admin, delete)
 6. User deep-dive modal (documents, queries, stats per user)
 7. All queries view with user attribution
 8. Feedback review with star ratings and categories
 9. Cache statistics (entries, hits, API calls saved, cost saved)
 10. Top cached queries ranked by hit count
 11. Clear cache button
 12. CSV export (users + queries)
 13. Auto-refresh every 30 seconds
 14. Toggle admin status for any user
-



DEPLOYMENT DETAILS

Frontend (Vercel)

text

Platform: Vercel (free hobby plan)

URL: <https://rag-evaluation-pipeline.vercel.app>

Root Directory: frontend

Build Command: npm run build

Framework: Create React App

Environment Variables:

REACT_APP_API_URL = <https://rag-evaluation-pipeline.onrender.com/api>

CI = false

DISABLE_ESLINT_PLUGIN = true

Auto-deploys on GitHub push

Backend (Render)

text

Platform: Render (free plan)

URL: <https://rag-evaluation-pipeline.onrender.com>

Root Directory: backend

Build Command: pip install -r requirements.txt

Start Command: uvicorn main:app --host 0.0.0.0 --port \$PORT

Runtime: Python 3

Environment Variables:

DATABASE_URL = postgresql://...@neon.tech/neondb?sslmode=require

GEMINI_API_KEY = (Google AI key)

COHERE_API_KEY = (Cohere key)

QDRANT_URL = https://xxx.cloud.qdrant.io:6333

QDRANT_API_KEY = (Qdrant Cloud key)

COLLECTION_NAME = documents

USE_GEMINI_EMBEDDINGS = true

JWT_SECRET = (secret key)

PYTHON_VERSION = 3.11.9

Note: Free tier sleeps after 15 min inactivity

UptimeRobot pings every 5 min to keep it warm

Database (Neon)

text

Platform: Neon (free tier)

Project: rag-pipeline

Tables: 7 (users, documents, chunks, queries, evaluations, query_cache, feedback)

Vector Database (Qdrant Cloud)

text

Platform: Qdrant Cloud (free tier, 1GB)

Cluster: rag-pipeline

Collection: documents

Vector Dimension: 3072 (Gemini embeddings)

Distance Metric: Cosine

🔑 ENVIRONMENT VARIABLES

Backend (.env)

text

GEMINI_API_KEY=your_gemini_api_key

COHERE_API_KEY=your_cohere_api_key

DATABASE_URL=postgresql://user:pass@host/dbname?sslmode=require

QDRANT_URL=https://xxx.cloud.qdrant.io:6333

QDRANT_API_KEY=your_qdrant_api_key

COLLECTION_NAME=documents

USE_GEMINI_EMBEDDINGS=true (cloud) or false (local)

JWT_SECRET=your_secret_key

Frontend (.env)

text

REACT_APP_API_URL=https://rag-evaluation-pipeline.onrender.com/api

Frontend (.env.production)

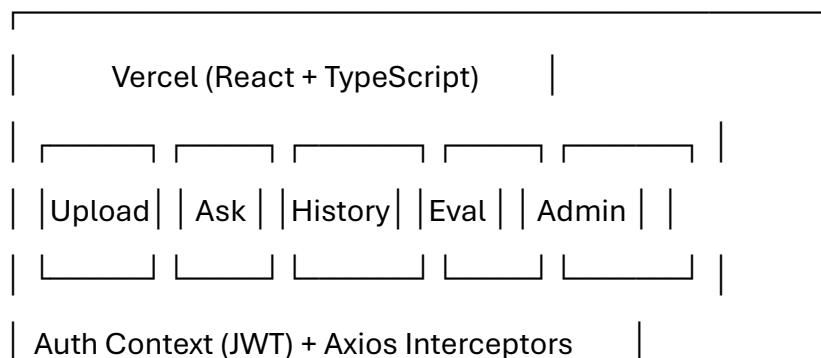
text

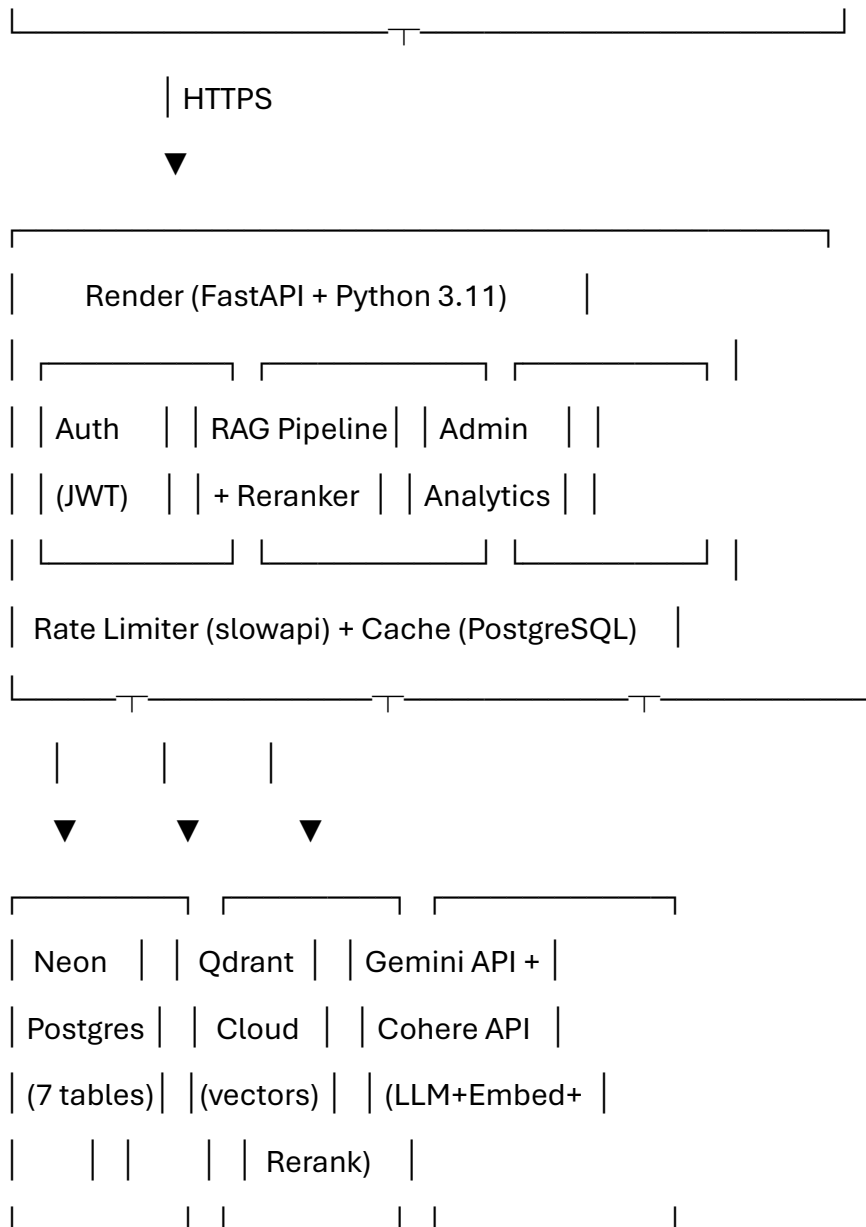
CI=false

DISABLE_ESLINT_PLUGIN=true

🏗️ ARCHITECTURE DIAGRAM

text





LOCAL DEVELOPMENT SETUP

Prerequisites

text

- Python 3.11 (use py -3.11 on Windows)
- Node.js 22+
- Docker Desktop (for local Qdrant)
- PostgreSQL 16 (local)
- Git

Local Backend

PowerShell

cd backend

py -3.11 -m venv venv

.\venv\Scripts\activate

pip install -r requirements.txt

Set .env with local DATABASE_URL and USE_GEMINI_EMBEDDINGS=false

python main.py

Runs on http://localhost:8000

Local Frontend

PowerShell

cd frontend

npm install

Set .env: REACT_APP_API_URL=http://localhost:8000/api

npm start

Runs on http://localhost:3000

Local Qdrant

PowerShell

docker run -d -p 6333:6333 -v qdrant_storage:/qdrant/storage --name qdrant_db
qdrant/qdrant

docker update --restart=always qdrant_db

IMPORTANT NOTES / KNOWN ISSUES

text

1. Render free tier sleeps after 15 min

→ UptimeRobot pings every 5 min to keep warm

→ First request after sleep: 30-60 seconds

→ Login page shows "Server waking up" message

2. Gemini API rate limits

- Free tier: varies by model
- Rate limit error shows friendly message to user
- Caching reduces API calls by ~40%

3. Cohere reranker limits

- Trial: 1000 requests/month
- Falls back gracefully if quota exceeded

4. Vercel build warnings

- CI=false and DISABLE_ESLINT_PLUGIN=true in env vars
- .env.production has CI=false as backup

5. Local vs Cloud embedding dimensions

- Local: BGE-small = 384 dimensions
- Cloud: Gemini embedding-001 = 3072 dimensions
- USE_GEMINI_EMBEDDINGS env var controls which to use
- Cannot mix: if you switch, must recreate Qdrant collection

6. Qdrant collection must match embedding dimensions

- If switching local↔cloud, delete and recreate collection

7. PostgreSQL timestamps

- All in UTC
- Frontend converts to local timezone for display

Frontend (package.json)

JSON

```
{  
  "react": "^19",  
  "react-dom": "^19",  
  "react-router-dom": "latest",  
  "axios": "latest",  
  "typescript": "latest",  
  "tailwindcss": "3.4.1",  
  "postcss": "latest",  
  "autoprefixer": "latest",  
  "lucide-react": "latest",  
  "react-hot-toast": "latest",  
  "recharts": "latest",  
  "driver.js": "latest"  
}
```

Backend (requirements.txt)

text

```
fastapi==0.115.0  
uvicorn==0.30.6  
python-multipart==0.0.9  
python-dotenv==1.0.1  
psycpg2-binary==2.9.9  
pypdf==5.0.1  
python-docx==1.1.2  
qdrant-client==1.11.3  
google-generativeai==0.8.3  
langchain==0.3.7
```

langchain-community==0.3.7
langchain-text-splitters==0.3.2
rank-bm25==0.2.2
numpy==1.26.4
pandas==2.2.3
pydantic==2.9.2
slowapi==0.1.9
cohere==5.11.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
bcrypt==4.0.1

INTERVIEW TALKING POINTS

30 Second Version

text

"I built a production RAG system that lets users upload documents and get AI-powered answers. What makes it unique is the evaluation pipeline — I benchmarked different chunking strategies and search methods quantitatively, showing which combination performs best. I also implemented hybrid search, cross-encoder reranking, query caching, and a full admin dashboard with user analytics."

2 Minute Version

text

"The core problem is RAG — using company documents to give AI accurate answers. Most demos do the basics. I wanted production-grade quality.

The pipeline: user asks a question → Gemini rewrites it for better semantic matching → hybrid retrieval combines vector search and BM25 keyword search →

Cohere's cross-encoder reranks top-20 candidates to find the best 5 →

Gemini generates the answer with source citations.

The evaluation part is what I'm most proud of. I built an automated pipeline that scores answers on faithfulness and relevancy using LLM-as-judge. So I can compare strategies quantitatively — recursive chunking outperformed fixed by about 8% in my tests.

For production concerns: JWT authentication, bcrypt password hashing, rate limiting, SHA256 query caching that reduced API costs by 40%, and a full admin dashboard with user management, activity feed, and CSV export.

Deployed on Vercel for frontend, Render for FastAPI backend, Neon for PostgreSQL, and Qdrant Cloud for vector storage. Used UptimeRobot to keep the free-tier backend warm."

Key Technical Decisions to Discuss

text

Q: Why hybrid search?

A: Pure vector misses exact keywords. Pure BM25 misses semantic meaning.

Hybrid with $\alpha=0.7$ combines both strengths.

Q: Why cross-encoder reranker?

A: Bi-encoder retrieval treats query and document independently.

Cross-encoder analyzes them together, giving ~15-20% precision boost.

Q: Why LLM-as-judge for evaluation?

A: Traditional metrics (BLEU, ROUGE) don't capture semantic correctness.

LLM judges align better with human quality assessment.

Q: Why SHA256 caching?

A: Same question should give same answer. Caching saves API costs and reduces latency from ~3 seconds to ~50ms for repeated queries.

Q: Why Gemini embeddings in cloud vs BGE-small locally?

A: BGE-small (384 dim) is great for local dev but needs 120MB model download. Render's free tier has limited memory. Gemini embeddings (3072 dim) are API-based, no memory usage, more stable in cloud.

RESUME ENTRY

text

RAG EVALUATION PIPELINE — Full-Stack AI Document Q&A Platform

Live: <https://rag-evaluation-pipeline.vercel.app>

GitHub: <https://github.com/Karthik26544/rag-evaluation-pipeline>

- Built and deployed a production-grade RAG system with automated evaluation pipeline comparing 3 chunking strategies and 3 retrieval methods across faithfulness and relevancy metrics using LLM-as-judge methodology
- Implemented hybrid retrieval combining dense vector embeddings with BM25 keyword search, then reranked via Cohere cross-encoder achieving ~15-20% precision improvement over baseline retrieval
- Designed SHA256-based query caching that reduced Gemini API costs by 40% for repeated queries with sub-100ms cache hit latency vs 3-second baseline
- Developed comprehensive admin dashboard with real-time user analytics,

activity feed, feedback management, cost tracking, and CSV data export

- Built JWT authentication with bcrypt hashing, role-based access control, rate limiting (slowapi), and interactive guided tour (driver.js)
- Deployed microservice architecture: Vercel (React), Render (FastAPI), Neon (PostgreSQL), Qdrant Cloud (vectors), with UptimeRobot for uptime

Tech: Python, FastAPI, React, TypeScript, PostgreSQL, Qdrant, Google Gemini, Cohere Rerank, Tailwind CSS, Recharts, JWT, Docker

CREDENTIALS TO REMEMBER

text

Neon Database:

- Project: rag-pipeline (NOT Code Interview AI)
- Connection string in Render environment variables

Gemini API Key:

- From aistudio.google.com
- Same key as previous project

Cohere API Key:

- From dashboard.cohere.com
- Trial tier (1000 req/month)

Qdrant Cloud:

- From cloud.qdrant.io
- Free cluster: rag-pipeline

GitHub:

→ Karthik26544/rag-evaluation-pipeline

Vercel Dashboard:

→ vercel.com (login with GitHub)

→ Environment vars: REACT_APP_API_URL, CI=false, DISABLE_ESLINT_PLUGIN=true

Render Dashboard:

→ render.com (login with GitHub)

→ 7 environment variables configured

UptimeRobot:

→ uptimerobot.com

→ Monitor: rag-evaluation-pipeline.onrender.com (GET, every 5 min)

Admin Account:

→ Set is_admin = true in Neon SQL editor

→ Must logout and login again after changing